GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	SISTEMAS DISTRIBUIDOS				
TÍTULO DE LA PRÁCTICA:	Los Hilos (Threads)				
NÚMERO DE PRÁCTICA:	**01**	**AÑO LECTIVO:**	2026	**NRO. SEMESTRE:**	2026A
TIPO DE PRÁCTICA:	**INDIVIDUAL**				
	GRUPAL	**X**	**MÁXIMO DE ESTUDIANTES**		**4**
FECHA INICIO:	**13/04/2026**	**FECHA FIN:**	**17/04/2026**	**DURACIÓN:**	**2 horas**
RECURSOS A UTILIZAR: VSCode, Netbeans, Eclipse.					
DOCENTE(s): Mg. Maribel Molina Barriga					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: • Conocer y utilizar efectivamente los componentes de hilos para aplicaciones en las que se requiera y de manera adecuada.	
TEMAS: • Hilos (Threads)	
COMPETENCIAS	C.a Aplica de forma transformadora conocimientos de matemática, computación e ingeniería como herramienta para evaluar, sintetizar y mostrar información como fundamento de sus ideas y perspectivas para la resolución de problemas.

CONTENIDO DE LA GUÍA

I. MARCO CONCEPTUAL

Un hilo (Thread) es un proceso en ejecución dentro de un programa

Puede haber varios hilos en ejecución simultánea

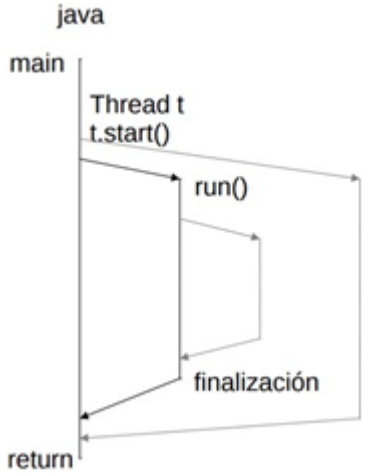
Los hilos implementan prioridad y mecanismos de sincronización

La finalización depende del hilo

Cualquier clase se puede hacer hilo

extends Thread

implements Runnable



Un proceso es una entidad que posee 2 características importantes:
Recursos: Básicamente: un espacio de direcciones (programas, datos y pila y un PCB), archivos, memoria, etc. El SOP realiza la función de protección para evitar interferencias no deseadas entre procesos en relación con los recursos.
Planificación/Ejecución: El proceso sigue una ruta de ejecución. Tiene un PC, un Estado de ejecución (Listo, bloqueado, ejecutándose, etc.) y una prioridad.

Estas dos características son independientes y pueden ser tratadas como tales por los sistemas de operación. En algunos sistemas de operación se le denomina a la unidad activa hilo (thread) y a la unidad propietaria de recursos se le suele denominar proceso o tarea.

EN UN ENTORNO MULTIHILO SE LE ASOCIA A LOS PROCESOS:

- Un espacio de direcciones virtuales que soporta la imagen del proceso.
- Acceso protegido a procesadores, otros procesos, archivos y recursos de E/S
- En un entorno multihilo se asocian a cada hilo:
- Un estado de ejecución.
- Un PC, un contexto (conjunto de registros) que se almacena cuando no está en ejecución.
- Una pila.
- Un espacio de almacenamiento para variables locales.
- Acceso a la memoria y recursos del proceso.

Para que la comunicación entre procesos sea posible es necesario usar las llamadas al sistema. Los threads comparten el mismo espacio de direcciones. Por lo tanto, el cambio de contexto entre un thread y otro que pertenecen al mismo proceso pudiera hacerse de forma totalmente independiente del sistema de operación. Los cambios de contexto de procesos son más costosos. Implican salvar el contexto del proceso, cambio de modo(trap al sistema de operación, etc), otro cambio de modo, restaurar contexto del nuevo proceso.

IMPLEMENTACIONES DE HILOS:

Threads a nivel de usuario.
Threads a nivel de kernel
Enfoque combinado, procesos Livianos

Usando hilos, se puede permitir el uso de llamadas al sistema bloqueantes sin necesidad de "bloquear" todo el proceso. Esta propiedad vuelve a los hilos particularmente atractivos para su uso dentro de sistemas distribuidos. Concentrémonos en la arquitectura Cliente Servidor.

CLIENTES MULTIHILOS

Sirven para esconder la latencia de comunicación a través de la red.

Ejemplo 1: Navegadores WEB

En muchos casos una página WEB consiste de un texto plano con múltiples figuras. Con frecuencia el navegador, establece la conexión con el servidor, recupera y comienza a desplegar la página HTML (incluso se permite al usuario el desplazamiento dentro de la página) mientras el navegador continúa recuperando otros archivos que conforman la página.
Desarrollar navegadores multihilos simplifica este hecho de forma considerable. Tan pronto como llega la página principal se pueden activar hilos que se encarguen de recuperar las demás partes. Cada hilo establece su propia conexión con el servidor. Mientras tanto el usuario advierte el retardo en las imágenes, pero puede ir explorando el documento.
Si el servidor está saturado o es lento no se observarán mejoras notables en el rendimiento.

Ejemplo 2: Servidores WEB Replicados

En muchos casos los servidores se replican en distintas máquinas y cada servidor proporciona el mismo conjunto de documentos WEB. Están localizados en el mismo sitio y se conocen por el mismo nombre.

Cuando entra una petición para una página WEB es re-enviada a uno de los servidores (usando round-robin u otra técnica de balanceo de carga).
Cuando se usan clientes multihilos cada conexión puede ir a una réplica diferente del mismo servidor. En este caso los distintos archivos se transmiten en paralelo asegurando que la página WEB completa se despliega en un tiempo más corto

El principal uso de la tecnología multihilos está del lado del servidor. Básicamente buscan mejorar el desempeño (aún en servidores monoprocesador) y la forma cómo se estructura el servidor.

Ejemplo: Por lo general un servidor de archivos espera una petición de entrada para una operación de archivo, posteriormente ejecuta la petición (operación bloqueante al disco) y luego envía la respuesta de regreso.
Modelo Servidor/Trabajador: Las peticiones son enviadas por los clientes hasta el servidor. Después de examinar la petición el hilo servidor elige un hilo trabajador sin utilizar y le encarga la petición.

El hilo trabajador realiza la lectura, lo cual puede provocar que se suspenda hasta que los datos sean recuperados. Si el hilo se suspende, el procesador, selecciona otro para su ejecución. ***¿Cómo se programa un servidor de archivos en la ausencia de hilos?*** El servidor recibe a peticiones, las examina y las trata de resolver antes de recibir la siguiente petición. Mientras espera por el disco el servidor está ocioso y no procesa otra petición. Se procesan menos peticiones por segundo (throughput).

Con los hilos se gana un rendimiento considerable. Cada hilo se programa secuencialmente en forma tradicional.

Supongamos que los hilos no están disponibles, pero los diseñadores del sistema consideran inaceptable el rendimiento debido al uso de un solo hilo y operaciones bloqueantes.

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

Caso Estudio: simular el proceso de cobro de un supermercado

Unos clientes van con un carro lleno de productos y una cajera les cobra los productos, pasándolos uno a uno por el escáner de la caja registradora.

En este caso la cajera debe de procesar la compra cliente a cliente, es decir que primero le cobra al cliente 1, luego al cliente 2 y así sucesivamente.

Para ello vamos a definir una clase "Cajera" y una clase "Cliente" el cual tendrá un "array de enteros" que representaran los productos que ha comprado y el tiempo que la cajera tardará en pasar el producto por el escáner; es decir, que si tenemos un array con [1,3,5] significará que el cliente ha comprado 3 productos y que la cajera tardara en procesar el producto 1 '1 segundo', el producto 2 '3 segundos' y el producto 3 en '5 segundos', con lo cual tardara en cobrar al cliente toda su compra '9 segundos'.

Código en Anexo 1:

Explicado este ejemplo vamos a ver cómo hemos definido estas clases:

- **Clase "Cajera.java"**
- **Clase "Cliente.java"**
- **Clase "Main.java":**

Como vemos se procesa primero la compra del cliente 1 y después la compra del cliente 2 tardando en procesar ambas compras un tiempo de 26 segundos.

¿Y si en vez de procesar primero un cliente y después otro, procesásemos los dos a la vez?, ***¿Cuánto tardaría el programa en ejecutarse?.*** Pues bien si en vez de haber solo una Cajera (es decir un solo hilo), hubiese dos Cajeras (es decir dos hilos o threads) podríamos procesar los dos clientes a la vez y tardar menos tiempo en ejecutarse el programa.

Para ello debemos de modificar la clase "Cajera.java" y hacer que esta clase herede de la clase Thread para heredar y sobre-escribir algunos de sus métodos. Primero vamos a ver como codificamos esta nueva clase "CajeraThread.java" y después explicamos sus características.

Lo primero que vemos y que ya hemos comentado es que la clase "CajeraThread" debe de heredar de la clase Thread: "extendsThread".

Otra cosa importante que vemos es que hemos sobre-escrito el método "run()" (de ahi la etiqueta @Override). Este método es imprescindibles sobre-escribirlo (ya que es un método que está en la clase Runnable y la clase Thread Implementa esa Interface) porque en él se va a codificar la funcionalidad que se ha de ejecutar en un hilo; es decir, que lo que se programe en el método "run()" se va a ejecutar de forma secuencial en un hilo.

En esta clase "CajeraThread" se pueden sobre-escribir más métodos para que hagan acciones sobre el hilo o thread como, por ejemplo, parar el thread, ponerlo en reposos, etc

- **Clase CajeraThread.java**

A continuación vamos a ver como programamos el método Main para que procese a los clientes de forma paralela y ver como se tarda menos en procesar todo. El método Main esta en la clase "MainThread.java" que tiene el siguiente contenido:

- **Clase "MainThread.java"**

Ahora vamos a ver cuál sería el resultado de esta ejecución y vamos a comprobar como efectivamente el programa se ejecuta de forma paralela y tarda solo 15 segundos en terminar su ejecución.

Otra forma de hacer lo mismo, pero sin heredar de la clase "Thread" es implementar la Interface "Runnable". En este caso no dispondremos ni podremos sobre-escribir los métodos de la clase Thread ya que no la vamos a utilizar y solo vamos a tener que sobre-escribir el método "run()". En este caso solo será necesario implementar el método "run()" para que los procesos implementados en ese método se ejecuten en un hilo diferente. Vamos a ver un ejemplo de cómo utilizando objetos de las clases "Cliente.java" y "Cajera.java" podemos implementar la multitarea en la misma clase donde se llama al método Main de la aplicación. A continuación, vemos la codificación en la clase:

- **Clase "MainRunnable.java"**

El concepto de multitarea o multiprocesamiento es bastante sencillo de entender ya que solo consiste en hacer varias cosas a la vez sin que se vea alterado el resultado final. Como ya se ha dicho en la entrada no todo se puede paralelizar y en muchas ocasiones suele ser complicado encontrar la manera de paralelizar procesos dentro de una aplicación sin que esta afecte al resultado de la misma, por tanto, aunque el concepto sea fácil de entender el aplicarlo a un caso práctico puede ser complicado para que el resultado de la aplicación no se vea afectado.

III. EJERCICIOS/PROBLEMAS PROPUESTOS

a. Implementar, ejecutar y probar el código de los anexos 2, de manera adecuada.
b. Evaluar resultados obtenidos.
c. Escriba un reporte sobre las tareas realizadas y resultados.
d. Escriba sus conclusiones

Caso Estudio: Productor - Consumidor

El productor genera un entero entre 0 y 9 (inclusive), lo almacena en un objeto "*CubbyHole*", e imprime el número generado. Para hacer más interesante el problema de la sincronización, el productor duerme durante un tiempo aleatorio entre 0 y 100 milisegundos antes de repetir el ciclo de generación de números.

```
class Productor extends Thread {
privateCubbyHole cubbyhole; private
int numero;
          public Productor(CubbyHole c, int
       numero) {cubbyhole = c; this.numero =
       numero;
     }
     public void run() {
       for (int i = 0; i < 10;
        i++) {cubbyhole.put(i);
        System.out.println("Productor  #"+this.numero+"
   pone:" +i);
            try {
```

```
                    sleep((int)(Math.random() * 100));
                } catch (InterruptedException e) {
                        }
        }
      }
    }
```

El consumidor, por su parte, está "hambriento", consume todos los enteros de `CubbyHole` (exactamente elmismo objeto en que el productor puso los enteros en primer lugar) tan pronto como estén disponibles.

```
class Consumidor extends Thread {
 privateCubbyHole cubbyhole; private
 int numero;

 public Consumidor(CubbyHole c, int numero) { cubbyhole

  = c;
  this.numero = numero;
 }
 public void run() { int value
  = 0;
  for (int i = 0; i < 10; i++) {
   value =cubbyhole.get();
   System.out.println("Consumidor  #"+this.numero+"  obtiene: "+value);
   }
  }
}
```

En este ejemplo, el Productor y el Consumidor comparten datos a través de un objeto `CubbyHole` común. Observará que ninguno de los dos hace ningún tipo de esfuerzo para asegurarse de que el consumidor obtiene cada valor producido una y sólo una vez. La sincronización entre estos dos hilos realmente ocurre a un nivel inferior, dentro de los métodos `get()` y `put()` del objeto `CubbyHole`. Sin embargo, asumamos por un momento que estos dos hilos no están sincronizados y veamos los problemas potenciales que podría provocar esta situación.

Un problema sería el que se daría cuando el Productor fuera más rápido que el Consumidor y generara dos números antes de que el Consumidor tuviera una posibilidad de consumir el primer número. Así el Consumidor se saltaría un número. Parte de la salida se podría parecer a esto.

```
Consumidor #1 obtiene: 3
Productor #1 pone: 4
Productor #1 pone: 5
Consumidor #1 obtiene: 5
    . . .
```

Otro problema podría aparecer si el consumidor fuera más rápido que el productor y consumiera el mismo valor dos o más veces. En esta situación el Consumidor imprimirá el mismo valor dos veces y podría producir una salida como esta.

```
    . . .
Productor #1 pone: 4
Consumidor #1 obtiene: 4
Consumidor #1 obtiene: 4
Productor #1 pone: 5
    . . .
```

De cualquier forma, el resultado es erróneo. Se quiere que el consumidor obtenga cada entero producido por el productor y sólo una vez. Los problemas como los descritos anteriormente, se llaman "condiciones de carrera". Se alcanzan cuando varios hilos ejecutados asíncronamente intentan acceder a un mismo objeto al mismo tiempo y obtienen resultados erróneos.

Para prevenir estas condiciones en nuestro ejemplo Productor/Consumidor, el almacenamiento de un nuevo entero en `CubbyHole` por el Productor debe estar sincronizado con la recuperación del entero por parte del Consumidor. El Consumidor debe consumir cada entero exactamente una vez. El programa productor-consumidor utiliza dos mecanismos diferentes para sincronizar los hilos Productor y Consumidor; los monitores, y los métodos `notify()` y `wait`.

Caso Estudio del productor-consumidor modificado

En el ejemplo anterior del productor- consumidor tiene dos métodos de sincronización: put(), que cambia el valor de *`CubbyHole`*, y `get()`, para recuperar el valor actual. Este sería el código fuente del objeto CubbyHole utilizando las técnicas de sincronización nuevas:

```
class  CubbyHole  {
    private int
    contents;
    private boolean available = false;

    public synchronized int get() {
        while(available == false) {
            try { wait();
            } catch (InterruptedException e) {
            }
        }
        available =
        false;
        notify();
        return contents;
    }

    public synchronized void put(int value) {
        while(available == true) {
            try {
                wait();
            } catch (InterruptedException e) {
            }
        }
        contents = value; available
        = true; notify();
    }
}
```

La variable `contents` tiene el valor actual de CubbyHole y available indica si se puede recuperar o no el valor. Cuando `available` es verdadero, el productor aún no ha acabado de producir.

`CubbyHole` tiene dos métodos de sincronización, y Java proporciona un solo monitor para cada ejemplar de `CubbyHole` (incluyendo el compartido por el Productor y el Consumidor). Siempre que el control entra en un método sincronizado, el hilo que ha llamado al método adquiere el monitor del objeto

al cual pertenece el método. Otros hilos no pueden llamar a un método sincronizado del mismo objeto mientras el monitor no sea liberado.

Nota: Java permite re-adquirir un monitor. En este caso se trata de los llamados monitores re-entrantes. Cuando el Productor invoca el método `put()` de `CubbyHole`, adquiere el monitor del objeto CubbyHole y por lo tanto el Consumidor no podrá llamar a `get()` de *`CubbyHole`* y se quedará bloquedo (existe un método, wait(), que libera temporalmente el monitor). De igual forma sucede cuando el Consumidor invoca get().

```
public synchronized void put(int value) {
        // El productor adquiere el monitor
    while(available == true) {
        try { wait();
        } catch (InterruptedException e) {
        }
    }
    contents = value; available
    = true;
    notify();// lo notifica al Productor
        // El productor libera el monitor
}
public synchronized int get() {
        // El consumidor adquier el
    monitorwhile (available == false) {
    try {
          wait();  //  espera  que  el  Productor  invoque  a
notify ()
        } catch (InterruptedException e) {
        }
    }
    available = false;
    notify();    return contents;
        // el Consumidor libera el monitor
}
```

IV. CUESTIONARIO

a. ¿Por qué es importante el estudio de hilos y multihilos en un sistema distribuido?
b. Describe ¿Cómo están compuestos los hilos? y ¿Cuál es la diferencia entre hilos y procesos?
c. Realice un cuadro comparativo de ventajas y desventajas del uso de hilos

V. REFERENCIAS Y BIBLIOGRÁFIA RECOMENDADAS:

[1] Tanenbaum, A.S. (2008). Sistemas distribuidos: principios y paradigmas. México. Pearson Educación.
[2] Ceballos, F. J. (2006). Java 2, Curso de programación. México: Alfaomega, RaMa.
[3] Deitel, H. M., & Deitel, P. J. (2004). Cómo programar en Java. México: Pearson ducación.
[4] García Tomás, J., Ferrando, S., & Piattini, M. (2001). Redes para procesos distribuidos. México: AlfaomegaRa-Ma.
[5] Orfali, R., & Harkey, D. (1998). Client/Server Programming with Java and CORBA. USA: Wiley
[6] programación multihilo https://oscarmaestre.github.io/servicios/textos/tema2.html
[7] Threads en Java https://lc.fie.umich.mx/~rochoa/Materias/PROGRAMACION/PROGRAMACION_2/HILOS.pdf
[8] https://biblus.us.es/bibing/proyectos/abreproy/11320/fichero/Capitulos%252F13.pdf

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN	
TÉCNICAS: *Problemas /Ejercicios propuestos / Preguntas formuladas / Resolución de casos*	**INSTRUMENTOS:** *Lista de cotejo*
CRITERIOS DE EVALUACIÓN • Identifica el proceso de hilos (Threads) • Utiliza las clases miembro de hilos de la biblioteca Java	

Anexo 1:

```
public class cajera {

	private String nombre;

	public cajera() {
	}

	public cajera(String nombre) {
		this.nombre = nombre;
	}

	public String getNombre() {
		return nombre;
	}

	public void setNombre(String nombre) {
		this.nombre = nombre;
	}

	public void procesarCompra(Cliente cliente, long timeStamp) {

		System.out.println("La cajera " + this.nombre +
				" COMIENZA A PROCESAR LA COMPRA DEL CLIENTE " + cliente.getNombre() +
				" EN EL TIEMPO: " + (System.currentTimeMillis() - timeStamp) / 1000 	+
				"seg");

		for (int i = 0; i < cliente.getCarroCompra().length; i++) {
			this.esperarXsegundos(cliente.getCarroCompra()[i]);
			System.out.println("Procesado el producto " + (i + 1) +
					" ->Tiempo: " + (System.currentTimeMillis() - timeStamp) / 1000 +
					"seg");
		}

		System.out.println("La cajera " + this.nombre + " HA TERMINADO DE PROCESAR " +
								cliente.getNombre() + " EN EL TIEMPO: " +
								(System.currentTimeMillis() - timeStamp) / 1000 + "seg");

	}
```

```
        private void esperarXsegundos(int segundos) {
                try {
                        Thread.sleep(segundos * 1000);
                } catch (InterruptedException ex) {
                        Thread.currentThread().interrupt();
                }
        }

}

public class CajeraThread extends Thread {

        private String nombre;

        private Cliente cliente;

        private long initialTime;


        public CajeraThread() {
        }

        public CajeraThread(String nombre, Cliente cliente, long initialTime) {
                this.nombre = nombre;
                this.cliente = cliente;
                this.initialTime = initialTime;
        }

        public String getNombre() {
                return nombre;
        }

        public void setNombre(String nombre) {
                this.nombre = nombre;
        }

        public long getInitialTime() {
                return initialTime;
        }

        public void setInitialTime(long initialTime) {
                this.initialTime = initialTime;
        }

        public Cliente getCliente() {
                return cliente;
        }

        public void setCliente(Cliente cliente) {
                this.cliente = cliente;
        }

        @Override
        public void run() {

                System.out.println("La cajera " + this.nombre + " COMIENZA A PROCESAR LA COMPRA DEL CLIENTE "
                                + this.cliente.getNombre() + " EN EL TIEMPO: "
                                + (System.currentTimeMillis() - this.initialTime) / 1000
                                + "seg");

                for (int i = 0; i < this.cliente.getCarroCompra().length; i++) {
                        // Se procesa el pedido en X segundos
                        this.esperarXsegundos(cliente.getCarroCompra()[i]);
                        System.out.println("Procesado el producto " + (i + 1)
                                        + " del cliente " + this.cliente.getNombre() + "->Tiempo: "
                                        + (System.currentTimeMillis() - this.initialTime) / 1000
                                        + "seg");
                }
```

```
                System.out.println("La cajera " + this.nombre + " HA TERMINADO DE PROCESAR "
                                                + this.cliente.getNombre() + " EN EL TIEMPO: "
                                                + (System.currentTimeMillis() - this.initialTime) / 1000
                                                + "seg");
        }

        private void esperarXsegundos(int segundos) {
                try {
                        Thread.sleep(segundos * 1000);
                } catch (InterruptedException ex) {
                        Thread.currentThread().interrupt();
                }
        }

}

public class Cliente  {

        private String nombre;
        private int[] carroCompra;

        public Cliente() {
        }

        public Cliente(String nombre, int[] carroCompra) {
                this.nombre = nombre;
                this.carroCompra = carroCompra;
        }

        public String getNombre() {
                return nombre;
        }

        public void setNombre(String nombre) {
                this.nombre = nombre;
        }

        public int[] getCarroCompra() {
                return carroCompra;
        }

        public void setCarroCompra(int[] carroCompra) {
                this.carroCompra = carroCompra;
        }

}

public class Main {

        public static void main(String[] args) {

                Cliente cliente1 = new Cliente("Cliente 1", new int[] { 2, 2, 1, 5, 2, 3 });
                Cliente cliente2 = new Cliente("Cliente 2", new int[] { 1, 3, 5, 1, 1 });

                cajera cajera1 = new cajera("Cajera 1");
                cajera cajera2 = new cajera("Cajera 2");

                // Tiempo inicial de referencia
                long initialTime = System.currentTimeMillis();

                cajera1.procesarCompra(cliente1, initialTime);
                cajera2.procesarCompra(cliente2, initialTime);
        }
}

public class MainRunnable implements Runnable{

        private Cliente cliente;
        private cajera cajera;
```

```
        private long initialTime;

        public MainRunnable (Cliente cliente, cajera cajera, long initialTime){
                this.cajera = cajera;
                this.cliente = cliente;
                this.initialTime = initialTime;
        }

        public static void main(String[] args) {

                Cliente cliente1 = new Cliente("Cliente 1", new int[] { 2, 2, 1, 5, 2, 3 });
                Cliente cliente2 = new Cliente("Cliente 2", new int[] { 1, 3, 5, 1, 1 });

                cajera cajera1 = new cajera("Cajera 1");
                cajera cajera2 = new cajera("Cajera 2");

                // Tiempo inicial de referencia
                long initialTime = System.currentTimeMillis();

                Runnable proceso1 = new MainRunnable(cliente1, cajera1, initialTime);
                Runnable proceso2 = new MainRunnable(cliente2, cajera2, initialTime);

                new Thread(proceso1).start();
                new Thread(proceso2).start();

        }

        @Override
        public void run() {
                this.cajera.procesarCompra(this.cliente, this.initialTime);
        }

}

public class MainThread {

        public static void main(String[] args) {

                Cliente cliente1 = new Cliente("Cliente 1", new int[] { 2, 2, 1, 5, 2, 3 });
                Cliente cliente2 = new Cliente("Cliente 2", new int[] { 1, 3, 5, 1, 1 });

                //Tiempo inicial de referencia
                long initialTime = System.currentTimeMillis();
                CajeraThread cajera1 = new CajeraThread("Cajera 1", cliente1, initialTime);
                CajeraThread cajera2 = new CajeraThread("Cajera 2", cliente2, initialTime);

                cajera1.start();
                cajera2.start();
        }
}
```

Anexo 2:

```
public class Consumidor extends Thread {
        private CubbyHole cubbyhole;
        private int numero;
        public Consumidor(CubbyHole c, int numero) {
        cubbyhole = c;
        this.numero = numero;
        }
        public void run() {
        int value = 0;
        for (int i = 0; i < 10; i++) {
        value = cubbyhole.get();
        System.out.println("Consumidor #"+this.numero+" obtiene:"+value);
        }
        }
```

```
        }
public class CubbyHole{ private int contents;
private boolean available = false;
public synchronized int get() {
while (available == false) {
try {
wait();
} catch (InterruptedException e) { }
}
available = false;
notifyAll();
return contents;
}
public synchronized void put(int value) {
        while (available == true) {
                try {
                wait();
                } catch (InterruptedException e) { }
                }
                contents = value;
                available = true;
                notifyAll();
                }
                }
public class Demo {
        public static void main(String[] args) {
                CubbyHole cub = new CubbyHole();
                Consumidor cons = new Consumidor(cub,1);
                Productor prod = new Productor(cub,1);

                prod.start();
                cons.start();
        }

}

public class Productor extends Thread {
        private CubbyHole cubbyhole;
        private int numero;
        public Productor(CubbyHole c, int numero) {
        cubbyhole = c;
        this.numero = numero;
        }
        public void run() {
        for (int i = 0; i < 10; i++) {cubbyhole.put(i);
        System.out.println("Productor #" + this.numero + "pone:" + i);
        try {
        sleep((int)(Math.random() * 100));
        }
        catch (InterruptedException e) { }
        }
        }
        }
```